

Métricas para Proyecto Web

1. Métricas de Eficiencia del Desarrollo

Estas métricas permitirán evaluar el desempeño del equipo y la productividad en el desarrollo del proyecto:

- **Velocidad de Desarrollo (Velocity):** Número de funcionalidades completas entregadas en cada entrega parcial.
 - **Método:** Usar un tablero de tareas (Trello, Jira, GitHub Projects) donde cada funcionalidad tenga una tarjeta.
 - **Cómo medirlo:** Contar el número de funcionalidades completas en cada entrega parcial.
 - **Ejemplo:** Si en la primera entrega parcial se completan 4 funcionalidades y en la segunda entrega se completan 5 más, la velocidad acumulada es de 9 funcionalidades.
- **Tiempo de Resolución de Tareas:** Tiempo promedio que tarda el equipo en completar una funcionalidad desde que se inicia hasta que se entrega.
 - **Método:** Registrar la fecha en que una tarea es creada y la fecha en que se marca como completada en el tablero de tareas.
 - **Cómo medirlo:** Promediar los días que tarda en completarse una tarea.
 - **Ejemplo:** Si una tarea se inicia el 10 de marzo y se finaliza el 12 de marzo, el tiempo de resolución es de 2 días.
- **Cumplimiento de Fechas de Entrega:** Si las entregas parciales y final se cumplen en el plazo establecido.
 - **Método:** Comparar la fecha real de entrega con la fecha planificada.
 - **Cómo medirlo:** Medir el porcentaje de tareas entregadas dentro del plazo.
 - **Ejemplo:** Si de 10 funcionalidades planificadas para una entrega parcial, solo se completaron 8 a tiempo, el cumplimiento es del 80%.
- **Cantidad de Commits por Semana:** Permite medir la constancia del trabajo en el repositorio de GitHub.
 - **Método:** Revisar el historial de commits en GitHub o GitLab.
 - **Cómo medirlo:** Contar el número de commits por semana y evaluar si hay actividad constante.

- **Ejemplo:** Si en la primera semana hubo 20 commits, en la segunda semana 15 y en la tercera 30, se obtiene una media de 21.67 commits por semana.
- **Cobertura del Código (Code Coverage):** Medir cuántas líneas de código están cubiertas por pruebas automatizadas.
 - **Método:** Usar herramientas de testing como Jest (para frontend) y Mocha o Chai (para backend).
 - **Cómo medirlo:** Revisar el porcentaje de líneas de código cubiertas por pruebas automatizadas.
 - **Ejemplo:** Si el backend tiene 1000 líneas de código y 800 están cubiertas por pruebas, la cobertura es del 80%.
- **Cantidad de Errores Detectados en Pruebas:** Número de bugs encontrados en cada iteración.
 - **Método:** Usar herramientas de pruebas automatizadas y manuales (Postman para APIs, Jest para frontend).
 - **Cómo medirlo:** Contar los errores detectados en cada fase de prueba antes de la entrega final.
 - **Ejemplo:** Si en la primera fase de pruebas se encontraron 10 errores y en la segunda 5, el total de errores es 15.

2. Métricas de Calidad del Software

Estas métricas ayudan a evaluar la estabilidad y el rendimiento del sistema:

- **Tiempo de Respuesta del Servidor:** Medir cuánto tarda en responder el backend (idealmente < 200ms).
 - **Método:** Usar herramientas como Postman o Chrome DevTools para medir el tiempo de respuesta de las API.
 - **Cómo medirlo:** Promediar el tiempo de respuesta de múltiples solicitudes.
 - **Ejemplo:** Si en 5 pruebas de la API `/login`, el tiempo de respuesta fue 150ms, 180ms, 200ms, 190ms y 170ms, el tiempo promedio es de 178ms.
- **Disponibilidad del Sistema:** Cuánto tiempo el sistema está operativo sin fallos.
 - **Método:** Usar herramientas como Pingdom o UptimeRobot para monitorear el sistema.
 - **Cómo medirlo:** Calcular el porcentaje de tiempo en que el sistema estuvo disponible.
 - **Ejemplo:** Si en un mes hubo 43 minutos de inactividad, la disponibilidad es:

$$\left(\frac{(30 \times 24 \times 60) - 43}{30 \times 24 \times 60} \right) \times 100 = 99.9\%$$

- **Pruebas de Carga y Escalabilidad:** Evaluar cuántos usuarios simultáneos soporta el sistema sin afectar el rendimiento.
 - **Método:** Usar herramientas como Apache JMeter o k6 para simular múltiples usuarios.
 - **Cómo medirlo:** Determinar cuántos usuarios simultáneos puede manejar el sistema antes de que el tiempo de respuesta supere un umbral (ej. 500ms).
 - **Ejemplo:** Si el sistema mantiene tiempos de respuesta óptimos con hasta 100 usuarios concurrentes pero se degrada con 150, la capacidad máxima es de 100 usuarios.
- **Seguridad del Sistema:** Revisar vulnerabilidades como inyecciones SQL, XSS y manejo de JWT.
 - **Método:** Realizar pruebas de seguridad con herramientas como OWASP ZAP o Burp Suite.
 - **Cómo medirlo:** Contar el número de vulnerabilidades encontradas.
 - **Ejemplo:** Si en una prueba de seguridad se detectan 3 inyecciones SQL y 2 vulnerabilidades XSS, el total de fallos de seguridad es 5.
- **Experiencia de Usuario (UX):** Evaluar si la UI es intuitiva y fácil de usar (se puede medir con encuestas o pruebas de usuario).
 - **Método:** Realizar encuestas a usuarios o pruebas de usabilidad.
 - **Cómo medirlo:** Promediar la calificación de satisfacción en una escala del 1 al 5.
 - **Ejemplo:** Si 10 usuarios califican la experiencia con puntajes de 4, 5, 4, 5, 4, 4, 5, 3, 4 y 5, el promedio es **4.3**.
- **Tasa de Éxito de las Reservaciones:** Número de intentos de reservación completados sin errores versus los intentos fallidos.
 - **Método:** Contar las reservaciones completadas versus los intentos fallidos.
 - **Cómo medirlo:** Dividir el número de reservaciones exitosas por el número total de intentos.
 - **Ejemplo:** Si de 500 intentos de reservación, 450 fueron exitosos y 50 fallaron, la tasa de éxito es:

$$\left(\frac{450}{500}\right) \times 100 = 90\%$$

- **Registro de Errores en Producción:** Cuántos errores inesperados ocurren cuando los usuarios interactúan con la aplicación.
 - **Método:** Usar herramientas como Sentry o logs en el servidor.
 - **Cómo medirlo:** Contar el número de errores registrados en el backend y frontend.
 - **Ejemplo:** Si en una semana se registraron 10 errores en el backend y 5 en el frontend, el total de errores es 15.

